

# Documentação de Contratos (API REST)

Projeto: Evolutto - Mapeamento de DTOs (Java & Angular)

Autor: Allan Victor Rodrigues Souza

*Nota Arquitetural: Para garantir que a comunicação entre o backend em Spring Boot e o frontend em Angular seja limpa, segura e tipada, esta documentação utiliza Records (Java 14+) para o backend e Interfaces no TypeScript. Isso evita a exposição direta das entidades do banco de dados (JPA) e reduz efeitos colaterais.*

# 1. Domínio Core: Hábitos e Execução

---

## Java (Spring Boot) - Records

```
// O que o Angular envia para criar um hábito
public record CreateHabitRequest(
    @NotBlank String title,
    String description,
    @NotNull HabitType type,
    @NotNull HabitDifficulty difficulty
) {}

// O que o Angular recebe ao listar hábitos
public record HabitResponse(
    UUID id,
    String title,
    String description,
    HabitType type,
    HabitDifficulty difficulty,
    boolean isActive
) {}

// O que o Angular recebe após o usuário dar "Check"
public record ExecuteHabitResponse(
    UUID logId,
    LogStatus status, // COMPLETED ou PENDING_APPROVAL
    int xpRewarded,
    int coinsRewarded,
    int newTotalXp,
    int newTotalCoins,
    int currentDebuffCounter
) {}
```

## 2. Domínio Core: Lojinha

---

### Java (Spring Boot) - Records

```
// O que o Angular envia para criar uma recompensa
public record CreateRewardRequest(
    @NotBlank String title,
    @Positive int cost
) {}

// O que o Angular recebe ao listar a loja
public record ShopResponse(
    ShopStatus status, // ACTIVE ou FROZEN
    List<RewardItem> availableRewards
) {}

public record RewardItem(
    UUID id,
    String title,
    int cost
) {}

// O que o Angular recebe após tentar comprar
public record BuyRewardResponse(
    boolean success,
    int newCoinBalance,
    String message
) {}
```

### 3. Domínio Social: Grupos e Leaderboard

---

#### Java (Spring Boot) - Records

```
public record CreateGroupRequest(  
    @NotBlank String name,  
    String description,  
    boolean isPrivate  
) {}  
  
// Resposta processada e retornada do Redis  
public record LeaderboardResponse(  
    UUID groupId,  
    String groupName,  
    List<LeaderboardPosition> ranking  
) {}  
  
public record LeaderboardPosition(  
    int position,  
    UUID userId,  
    String username,  
    int periodXp  
) {}
```

## 4. Domínio Parental: Aprovações e Castigos

---

### Java (Spring Boot) - Records

```
// Lista de atividades pendentes para o pai aprovar
public record PendingApprovalResponse(
    UUID logId,
    UUID childId,
    String childName,
    String habitTitle,
    LocalDateTime executedAt
) {}

// O que o Angular envia para aprovar/rejeitar
public record ReviewHabitRequest(
    @NotNull Boolean isApproved
) {}

// O que o Angular envia para criar uma missão de redenção
public record CreateRedemptionMissionRequest(
    @NotNull UUID childId,
    @NotBlank String title
) {}
```

## 5. Mapeamento no Frontend (Angular / TypeScript)

---

Interfaces equivalentes para inferência de tipo no HttpClient do Angular.

```
// Enums
export enum HabitType { GOOD = 'GOOD', BAD = 'BAD' }
export enum HabitDifficulty { EASY = 'EASY', MEDIUM = 'MEDIUM', HARD = 'HARD' }
export enum LogStatus { COMPLETED = 'COMPLETED', PENDING_APPROVAL =
'PENDING_APPROVAL', REJECTED = 'REJECTED' }
export enum ShopStatus { ACTIVE = 'ACTIVE', FROZEN = 'FROZEN' }

// Interfaces de Request
export interface CreateHabitRequest {
  title: string;
  description?: string;
  type: HabitType;
  difficulty: HabitDifficulty;
}

export interface ReviewHabitRequest {
  isApproved: boolean;
}

// Interfaces de Response
export interface HabitResponse {
  id: string; // UUID é string no TS
  title: string;
  description: string;
  type: HabitType;
  difficulty: HabitDifficulty;
  isActive: boolean;
}

export interface ExecuteHabitResponse {
  logId: string;
  status: LogStatus;
  xpRewarded: number;
  coinsRewarded: number;
  newTotalXp: number;
  newTotalCoins: number;
  currentDebuffCounter: number;
}

export interface ShopResponse {
  status: ShopStatus;
  availableRewards: RewardItem[];
}

export interface RewardItem {
  id: string;
  title: string;
```

```
cost: number;
```

```
}
```